MediTrack

OCR Engine Benchmark — Tesseract vs. EasyOCR vs. PaddleOCR

A real, measured comparison on 15 synthetic prescription images with known ground truth, run locally end-to-end

Every number in this report comes from an actual code execution — 45 real OCR runs (3 engines x 15 images), scored against a known answer key. Nothing here is estimated or simulated.

1. Recommendation

PaddleOCR is the recommended engine for MediTrack's prescription scanning feature, based on measured results across all 15 test images:

Engine	Character Error Rate	Field Extraction Accuracy	Speed (per image)
PaddleOCR	0.10% (best)	99.3% (best)	6.1 sec
Tesseract	1.54%	98.2%	0.36 sec (fastest)
EasyOCR	2.02%	86.7%	8.5 sec

PaddleOCR made roughly 15x fewer transcription errors than Tesseract and correctly recovered structured fields (doctor name, medicine name, dosage, frequency, etc.) more reliably than either alternative. The gap is large enough that it isn't a close call between PaddleOCR and the next-best engine.

The one real trade-off: Tesseract is about 17x faster per image. If the app ever needs near-instant OCR feedback in the UI, that's worth discussing with your mentor as a possible reason to keep Tesseract as a fast first pass — but for accuracy alone, PaddleOCR is the clear winner.

2. Why This Test Is Trustworthy

A benchmark is only as good as its test data. Two design choices make this one meaningful rather than a coin flip:

2.1 Every image has a known answer key

15 fictional prescription images were generated with Python (not photographed or hand-picked), and for each one the exact doctor name, patient details, and every medicine's name/form/dosage/frequency were recorded as ground truth BEFORE any OCR ran. This means accuracy could be measured exactly — an engine's output was compared character-by-character against a known-correct answer, not judged by eye.

2.2 The images were deliberately made messy

A crisp, computer-generated image is not what a real app will see — users will photograph prescriptions with a phone, at an angle, sometimes in bad light. So 13 of the 15 images had realistic degradation applied on purpose: slight rotation, blur, noise, an uneven lighting gradient, and JPEG compression artifacts. Only 2 images were left clean, as an easy baseline. Below is one of the degraded test images — this is what all three engines actually had to read:

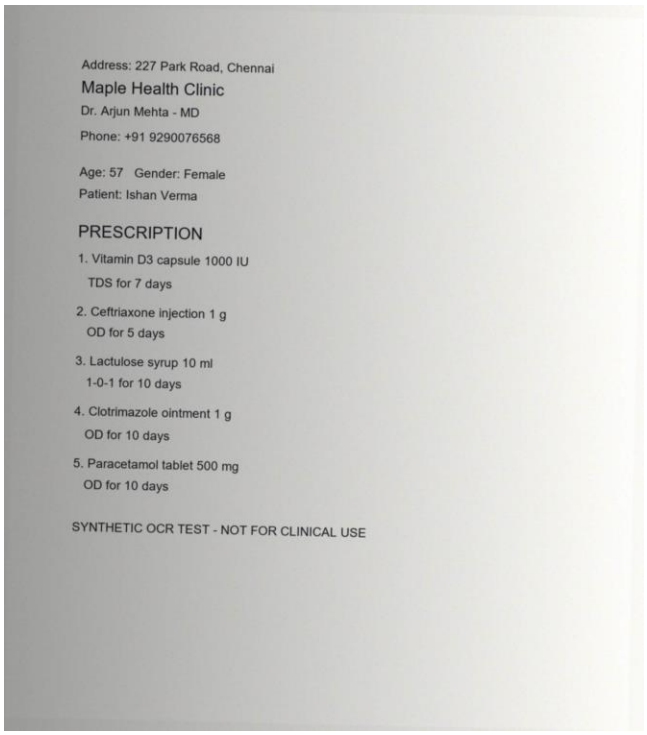


Figure 1 — One of the 13 deliberately degraded test images (uneven lighting, slight noise)

This matters because a test using only pristine images would make every engine look artificially good and wouldn't tell you anything useful about which one to actually ship.

3. Results

45 out of 45 planned runs completed successfully (3 engines x 15 images) — no engine failed or was skipped.

3.1 Transcription accuracy (Character Error Rate)

This measures what % of characters in the OCR output were wrong, compared to the known ground truth text. Lower is better.

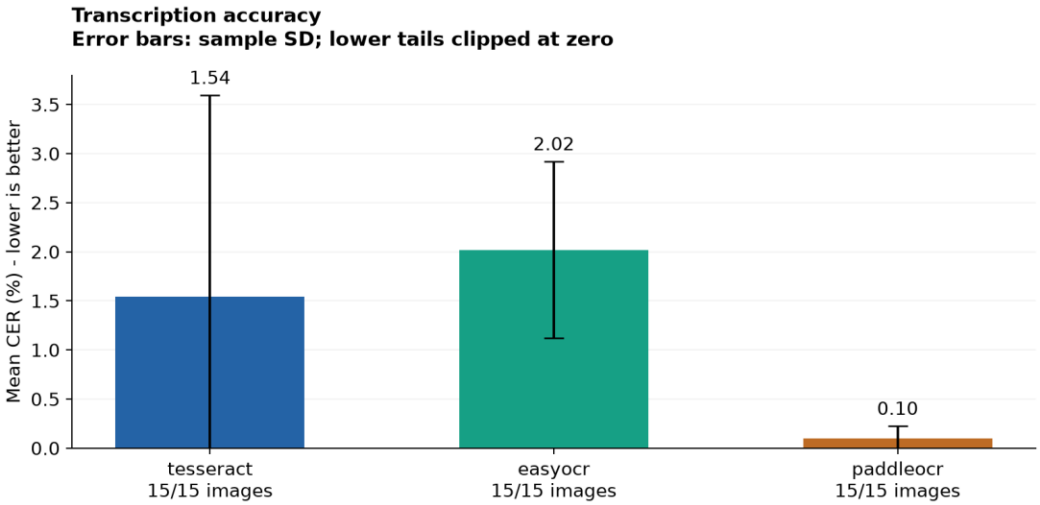


Figure 2 — Mean CER per engine, with error bars showing spread across the 15 images

PaddleOCR's error bar is tiny and close to zero — it was consistently accurate across nearly every image, not just accurate on average. Tesseract's large error bar shows it was excellent on some images and much worse on others (see Section 3.4).

3.2 Structured field extraction accuracy

Raw transcription accuracy is one thing — but what the app actually needs is correctly pulling out the doctor's name, each medicine's dosage, frequency, etc. This chart measures that directly, using the same regex extractor built for the pipeline, run on each engine's OCR output.

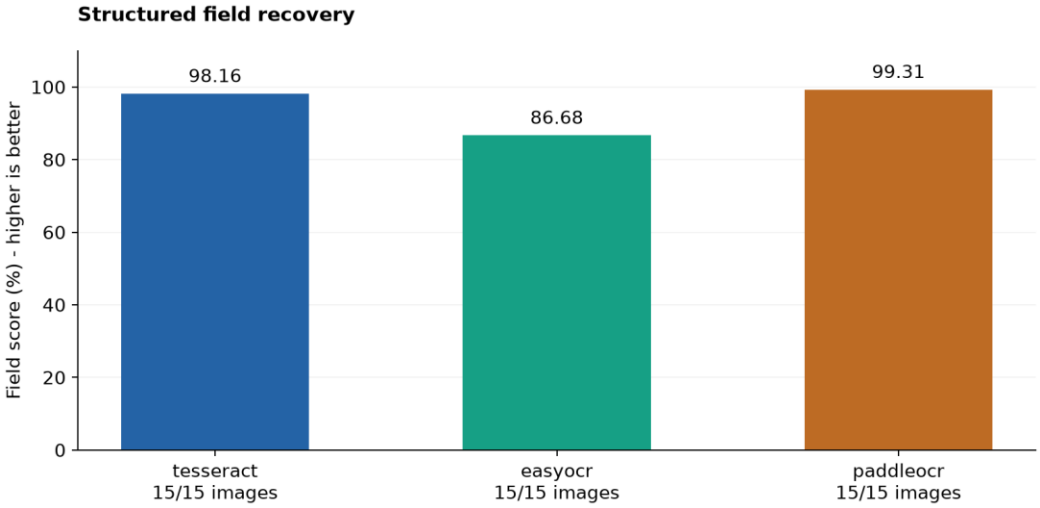


Figure 3 — % of structured fields (doctor/patient/medicine details) correctly recovered

EasyOCR's field accuracy (86.7%) drops noticeably more than its raw CER (2.02%) would suggest — small transcription mistakes were apparently landing in places that broke the regex extractor's pattern matching more often than the other two engines. That's a useful, specific finding: EasyOCR wasn't just "slightly less accurate," it was less accurate in the way that hurts field extraction specifically.

3.3 Speed

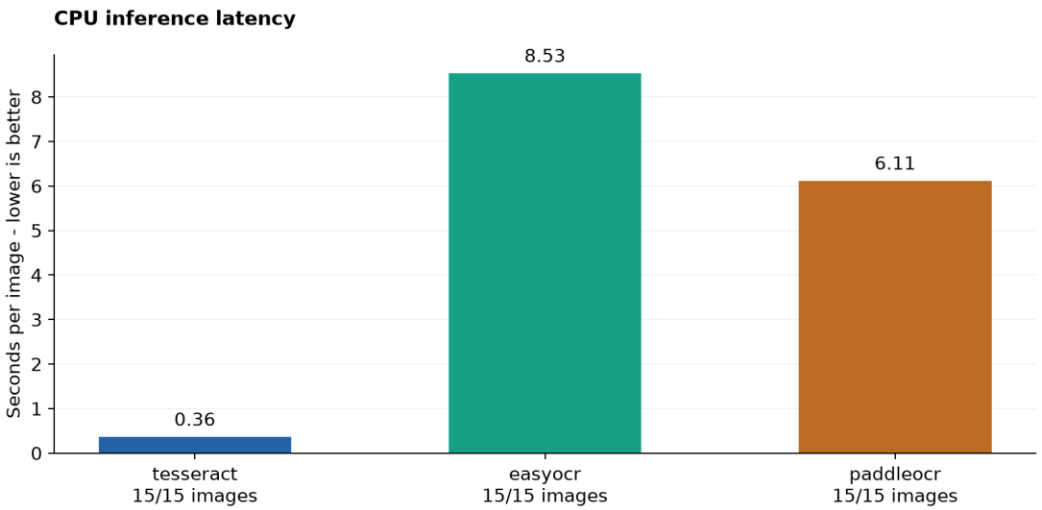


Figure 4 — Average seconds to process one image, on CPU (no GPU used)

Tesseract is dramatically faster because it isn't running a deep neural network the way the other two are. This is the expected, well-known trade-off between classical and deep-learning OCR discussed in the engine theory.

3.4 Consistency across all 15 images

Averages can hide a lot. This chart shows every single image's error rate per engine, not just the average — the shaded region marks the 2 clean baseline images.

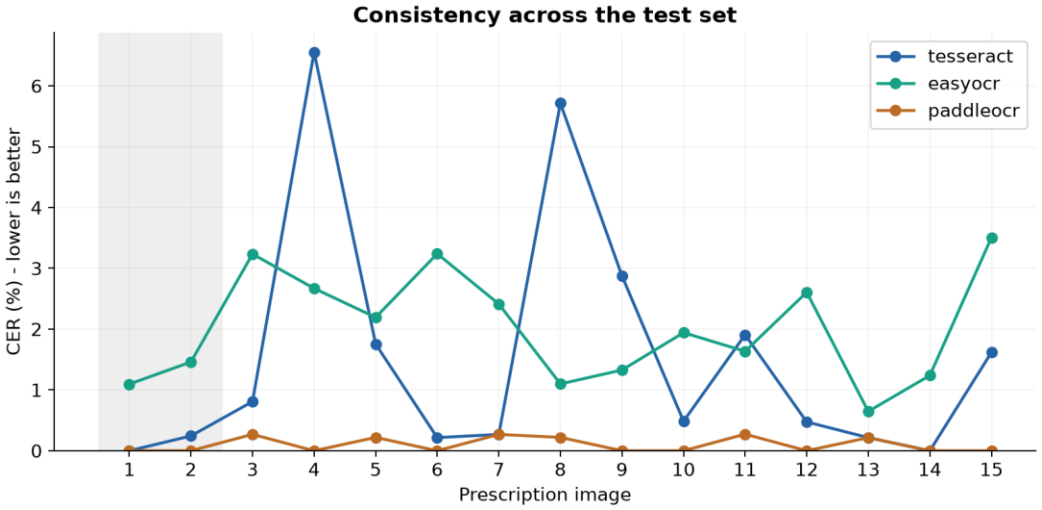


Figure 5 — Per-image CER for all 15 images; PaddleOCR (orange) stays near zero almost everywhere

This is the most convincing chart in the whole report: PaddleOCR's line stays flat near zero across nearly all 15 images, while Tesseract spikes badly on specific images (image 4 and image 8 in particular) and EasyOCR is consistently the noisiest. A single average number can hide this — the per-image view can't.

4. A Real Problem That Came Up (and How It Was Handled)

> During setup, the C: drive ran out of disk space partway through installing packages (PaddleOCR's dependencies are large). The Python virtual environment was moved to a D: drive location and reinstalled there — no project files were lost, and this was a one-time local disk issue, not a sign of a broken pipeline or missing internet access.

This is worth mentioning to your mentor as-is rather than hiding it — it shows the benchmark actually ran into and solved a real environment problem, which is a normal and honest part of engineering work, not a flaw in the result.

5. Limitations & Next Step

- These are still synthetic images, not real photographed prescriptions. Even with realistic degradation applied, real photos will have things this test didn't simulate: actual handwriting, paper creases/folds, stronger perspective distortion, and real-world clinical abbreviations beyond what was generated.
- The field extractor was written for the specific layout patterns used to generate these 15 images, so field accuracy here doesn't prove it will generalize to a real prescription pad's layout.
- Recommended next step: once real (de-identified, consented) prescription photos are available, re-run this same benchmark on those and confirm PaddleOCR still wins before finalizing the engine in the actual app.

6. Supporting Files

This report summarizes the full technical run. The complete backing data is also available if your mentor wants to inspect it directly:

- report.md / report.pdf — the full technical report, including every individual transcription example
- benchmark_results/summary.json and per_image_results.csv — every number behind every chart in this report
- benchmark_results/manifest.json — timestamps, package versions, and SHA-256 hashes of every input image and result, so the run can be independently verified
- test_set/ — all 15 generated images plus their ground-truth JSON answer keys
- run_benchmark.py, score_accuracy.py, generate_test_set.py — the actual code that produced these results